

System Test Plan

Team3-Basyx-Editor

Version Control

Version	Date	Author	Comment
1.0	16.04.2026	Daniel Ziegler	Version 1.0
1.1	24.04.2026	Daniel Ziegler	Fromulating Test Cases, adding Testfiles
1.2	05.05.2026	Daniel Ziegler	Linking the testcases to the right Requirements
1.3	06.05.2026	Daniel Ziegler	Fixed table of contents, corrected the testfile linkage
1.4	15.05.2026	Daniel Ziegler	Added external links, small format changes

Table of contents

1. [Introduction](#)

2. [Scope](#)

3. [Test Strategy](#)

4. [Test Environment](#)

◦ 4.1 [Hardware Environment](#)

◦ 4.2 [Software Environment](#)

◦ 4.3 [Deployment Setup](#)

5. [Test Cases Overview](#)

◦ 5.1 [TC01 Proper XML-Validation MOD01](#)

◦ 5.2 [TC02 Handle Error Messages MOD01](#)

◦ 5.3 [TC03 Display XML structure MOD02](#)

◦ 5.4 [TC04 AAS-Creation-Wizard functions properly MOD03](#)

◦ 5.5 [TC05 Extraction of Information for AAS-Generation MOD03](#)

6. [Test Data](#)

1. Introduction

The purpose of this System Test Plan (STP) is to define the overall testing approach for the BaSyx Editor Extension. It describes the test strategy, environment, test cases, and acceptance criteria required to verify that the system fulfills the requirements specified in the Software Architecture Specification ([SAS](#)).

2. Scope

This document covers testing of the changes planned in the BaSyx Editor Extension including:

- [MOD01](#) – File Import and Validation Subsystem

• [MOD02](#) – XML Viewer and Navigation Subsystem

• [MOD03](#) – AAS Generation and Submodel Mapping Subsystem

This document does not cover:

- Unit testing of minor changes to existing code

• Internal BaSyx backend functionality

3. Test Strategy

The testing strategy for the BaSyx Editor Extension ensures reliable validation of all added system functionalities through a structured and requirement-driven approach. A key objective is to achieve wide test coverage. Both positive and negative scenarios, including edge cases are considered to verify system robustness.

Testing is performed using a function isolation approach, where individual functionalities are tested independently. This improves defect detection and simplifies root cause analysis.

To validate requirements effectively, both file-based and code-based testing are applied. File-based testing uses input data (e.g., XML, KBL, VEC files) to verify the import, parsing, and visualization processes. Code-based testing uses Vitest, the existing testing environment the Basyx software uses, to ensure reliable, automated unit and integration test execution.

This approach ensures a comprehensive, traceable, and efficient validation of the system.

4. Test Environment

4.1. Hardware Environment

- Client machine (modern browser-capable system)

4.2. Software Environment

- Browser: Chrome, Firefox, Edge

• Frontend: Vue.js application

• Backend services:

-BaSyx Registry

-BaSyx AAS Repository

-BaSyx Submodel Service

4.3. Deployment Setup

- Docker-based deployment setup

5. Test Cases Overview

All testcases include:

- TC01 Proper XML-Validation MOD01

• TC02 Handle Error Messages MOD01

• TC03 Display XML structure MOD02

• TC04 AAS Creation Wizard functions properly MOD03

• TC05 Extraction of AAS Information MOD03

As an initial situation for all tests, it is assumed that the environment is functional and accessible and the BaSyx Web UI is open. Additionally, the mode is set to the BaSyx Editor, and at least one AAS is already imported and open. The AAS contains at least one Submodel with an XML-type file element included.

5.1. TC01 Proper XML-Validation MOD01

ID:	<TC01>
Name:	Proper XML-Validation
Req.-ID:	FR.02
Description:	The test case verifies that the function correctly validates XML, KBL, and VEC files by checking their headers and returns an appropriate error if the header is missing, invalid, or malformed.
Unit test	
Found in:	./tests/utlis/XMLValidator.test.ts

5.2. TC02 Handle Error Messages MOD01

--

5.2. TC02 Handle Error Messages MOD01

Name:	Handle Error Messages	
Req.-ID:	FR.01, FR.03	
Description:	The test case verifies that incorrect XML, KBL, and VEC files return consistent, accurate, and meaningful error messages.	
Test steps		
Step	Action	Expected result
1	Click the "add Submodel Element (+)" button within a Submodel	A dialog for the type of Element opens
2	Select "file" and click "next"	A dialog for element details opens
3	Under "Value" click "file" and add a file in the file input.	The file is added to the dialog, the ShortID is filled with the filename if empty. The MIME type is displayed correctly.
4	Optionally change other values including the ShortID	The changes are displayed
4	Click "save"	If the file is invalid, the file input is marked in red and an error is shown. If valid, an element is created with the correct MIME type.

5.3. TC03 Display XML structure MOD02

Test case		
ID:	<TC03>	
Name:	Display XML structure	
Req.-ID:	FR.04	
Description:	The test case verifies that the XML structure is correctly represented as a table of contents and that selecting entries navigates to the corresponding XML sections accurately and consistently within the viewer.	
Test steps		
Step	Action	Expected result
1	Choose a Submodel with a XML-type file element and open it.	A list of elements in the Submodel is shown.
2	Click on the XML-type file.	A detailed description of the Element opens on the right.
3	Click on "Visualization".	The XML preview with a table of contents and the XML content opens.
4	Scroll the XML content.	Only the XML content is scrolled.
5	Scroll the table of contents.	Only the table of contents is scrolled.
6	In table of contents click on an entry.	The XML content jumps to this entry and highlights the line.
7	In table of contents click on the triangle next to an entry.	The inner XML structure is revealed and the XML content does not jump to the entry.

5.4. TC04 AAS-Creation-Wizard functions properly MOD03

ID:	<TC04>	
Name:	AAS-Creation-Wizard functions properly	
Req.-ID:	FR.05	
Description:	The test case verifies that the creation wizard’s graphical interface correctly guides the user through the process, displaying appropriate inputs, navigation, and feedback when creating Asset Administration Shells from KBL or VEC files.	
Test steps		
Step	Action	Expected result
1	On the slide for the Asset Administration Shells, click on the three dots and on "Create AAS form KBL/VEC"	The wizard for the AAS creation opens
2	Click on the FileInput and choose a KBL or VEC file.	Only eligible files can be choosen. The file is displayed.
3	Optionally choose a picture for the AAS in the second FileInput.	Only eligible files can be choosen. The file is displayed.
4	Provide or generate AAS ID and Asset ID and Type. Optionally change general Information for the AAS that will be created	The fields for the Information are easy to use and display the entered values. The "generate" button fills the corresponding textinput with a generated value.
5	Choose Information which should be used to fill the Submodels of the AAS.	Individual Information can be selected or deselected.
6	Click "Create"	The AAS is created with the provided Information.

5.5. TC05 Extraction of Information for AAS-Generation MOD03

ID:	<TC05>
Name:	Extraction of the AAS Information
Req.-ID:	FR.06
Description:	The test case verifies that AAS information is correctly extracted from a given file, ensuring all relevant elements are accurately identified, parsed, and made available for further processing.
Unit Tests	
Found in:	./tests/utlis/KblVecUtils/KblVecDataPointTreeUtils.test.ts ./tests/utlis/KblVecUtils/KblVecRequiredFieldUtils.test.ts ./tests/utlis/KblVecUtils/KblVecSubmodelGenerationUtils.test.ts

6. Test Data

The test data consists of various files located in the '/Testfiles' folder. The test data includes data structures composed of shortened KBL and VEC files. Structural defects are included within otherwise valid structures.

Test data:	
Dataset	Data
1	_valid_kbl.kbl
2	_valid_svg.svg
3	_valid_vec.vec
4	_valid_xml.xml
5	kbl_faultykbl.kbl
6	kbl_faultyxml.kbl
7	kbl_noheader.kbl
8	kbl_nokbl.kbl
9	kbl_vec.vec
10	kbl_vec_inside.kbl
11	kbl_xml.xml
12	missing_close.vec
13	vec_endkbl.kbl
14	vec_kbl.kbl
15	vec_kbl_inside.vec
16	vec_noversion.vec